Set methods are listed below:

- s.add(e) - Adds element e to s.
- s.clear() - Removes all the elements of s.
- s.copy() - Shallow copy of s.
- s.discard(e) - Removes element e from s if it is present.
- s.pop() - Removes element and returns an element from s, raising KeyError if s is empty. However, the pop method removes the last element of the set and the set itself is not ordered. One needs to be cautious using this as any element could be removed.
- s.remove(e) - Removes element e from s, raising KeyError if e is not in s.
- s.isdisjoint(z) - Returns *True* if no elements are common in s and z.

▪ ***Sort***

sorted(s) gives the sorts the set values, and returns the ordered output. For example:

```
s = {3, 4, 2, 1}
print(sorted(s))

[1, 2, 3, 4]
```

▪ ***Set operators***

As the name suggests, sets are ideally useful for performing set operations. Set operations as infix operators are useful to make the code easier to read and analyse.

Examples are:

- s & z: s.__and__(z) - Intersection of s and z
- s | z: s.__or__(z) - Union of s and z
- s - z: s.__sub__(z) - Difference between s and z
- s ^ z: s.__xor__(z) - Symmetric difference
- e in s: s.__contains__(e) - Element e is present in s
- s < z: s.__le__(z) - Validates s is a subset of the z set
- s <= z: s.__lt__(z) - Validates s is proper subset of the z set
- s > z: s.__ge__(z) - Validates s is a super set of the z set
- s >= z: s.__gt__(z) - Validates s is proper super set of the z set
- s =& z: Intersection of s and z; s is updated

▪ ***Performance***

Here's a sample performance validation to check the values:

```
def return_unique_values_using_list(values):
   unique_value_list = []
   for val in values:
      if val not in unique_value_list:
          unique_value_list.append(val)
   return unique_value_list

def return_unique_values_using_set(values):
   unique_value_set = set()
   for val in values:
       unique_value_set.add(val)
   return unique_value_set

import time
id = [x for x in range(0, 100000)]
start_using_list = time.perf_counter()
return_unique_values_using_list(id)
end_using_list = time.perf_counter()
print("time elapse using list: {}".format(end_using_list -
start_using_list))

start_using_set = time.perf_counter()
return_unique_values_using_set(id)
end_using_set = time.perf_counter()
print("time elapse using set: {}".format(end_using_set -
start_using_set))

Output:
time elapse using list: 83.99319231199999
time elapse using set: 0.01616014200000393
```

As we can see, when it comes to the performance between set and list, set wins with a big margin. The time complexity for querying is O(1), while space complexity for storing is O(n).

Hash and hash tables

Hash and hash tables are two different but inter-related concepts.

- *Hash:* This is a function to generate a unique value for different sets of values.
- *Hash table:* This is a data structure to implement dictionaries and sets. Basically, it's a data structure to save keys and values.

▪ ***Hash***

A hash value for the given key to the hash function will always return the same hash value. The efficiency of the hash function lies in turning the key into an index of the list. Hence, an effective hash function and list can determine the index of data in the list without a search. Keys must be hashable objects.

For user defined dict class, if __eq__ method is implemented it requires __hash__ also to be implemented, as it should make a == b return *True* and hash(a) == hash(b) return *True* as well.

In Python 3, if you override __eq__, it automatically sets __hash__ to *None*, making the object unhashable. We need to manually override __hash__ to make it hashable again.

In Python we have a built-in hash function, which is used to generate a unique value for a different set of values. Here's an example of getting the same hash key running in